

LC 417: Pacific Atlantic Flow

Problem Overview & Rules

Goldman Sachs

Amazon

a water in a cell can flow to other adjacent cell only if their height is less than current cell

Bruteforce Approach

- for every cell run dfs
- can it reach pacific
- can it reach atlantic

Complexity: $O((n*m)^2)$

Optimal reverse Thinking

- which cells can pacific reach
- which cells can atlantic reach

→ reverse the stream

```
for (all pacific boundary cells) { dfs (or bfs) } for (all atlantic  
boundary cells) { dfs (or bfs) }
```

C++ Implementation

```
class Solution {
public:
    vector<vector<int>> pacificAtlantic(vector<vector<int>>& heights)
    {

        int n=heights.size();
        int m=heights[0].size();

        vector<vector<int>> seen1(n,vector<int>(m,0));
        vector<vector<int>> seen2(n,vector<int>(m,0));
        vector<vector<int>> ans;

        vector<pair<int,int>> dir={{-1,0},{1,0},{0,1},{0,-1}};
        queue<pair<int,int>> q;

        for(int i=0;i<n;i++){

            seen1[i][0]=1;
            q.push({i,0});
            while(!q.empty()){
                int r=q.front().first;
                int c=q.front().second;
                q.pop();
                for(auto it:dir){
                    int nr=r+it.first;
                    int nc=c+it.second;

                    if(nr>=0 && nc>=0 && nr<n && nc<m){
                        if(heights[nr][nc]>=heights[r][c] && !
seen1[nr][nc]){ //reverse stream jaa rhe isliye sign reverse
                            seen1[nr][nc]=1;
                            q.push({nr,nc});
                        }
                    }
                }
            }

            seen2[i][m-1]=1;
            q.push({i,m-1});
            while(!q.empty()){
                int r=q.front().first;
                int c=q.front().second;
                q.pop();
```

```

        for(auto it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(heights[nr][nc]>=heights[r][c] && !
seen2[nr][nc]){ //reverse stream jaa rhe isliye sign reverse
                    seen2[nr][nc]=1;
                    q.push({nr,nc});
                }
            }
        }
    }
}

```

//now lets handle top and bottom

```

for(int j=0;j<m;j++){

    seen1[0][j]=1;
    q.push({0,j});
    while(!q.empty()){
        int r=q.front().first;
        int c=q.front().second;
        q.pop();
        for(auto it:dir){
            int nr=r+it.first;
            int nc=c+it.second;

            if(nr>=0 && nc>=0 && nr<n && nc<m){
                if(heights[nr][nc]>=heights[r][c] && !
seen1[nr][nc]){ //reverse stream jaa rhe isliye sign reverse
                    seen1[nr][nc]=1;
                    q.push({nr,nc});
                }
            }
        }
    }

    seen2[n-1][j]=1;
    q.push({n-1,j});
    while(!q.empty()){
        int r=q.front().first;
        int c=q.front().second;
        q.pop();
        for(auto it:dir){

```

```

        int nr=r+it.first;
        int nc=c+it.second;

        if(nr>=0 && nc>=0 && nr<n && nc<m){
            if(heights[nr][nc]>=heights[r][c] && !
seen2[nr][nc]){ //reverse stream jaa rhe isliye sign reverse
                seen2[nr][nc]=1;
                q.push({nr,nc});
            }
        }
    }
}

for(int i=0;i<n;i++){
    for(int j=0;j<m;j++){
        if(seen1[i][j] && seen2[i][j]){
            ans.push_back({i,j});
        }
    }
}

return ans;
}
};

```